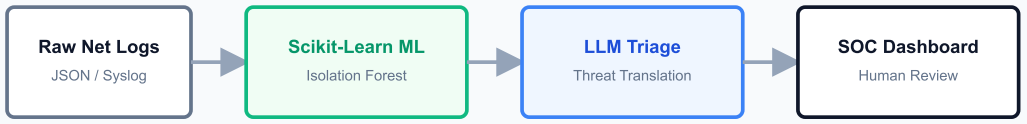
Aegis AI: Autonomous Tier-1 SOC Analyst

Target Architecture: Python (Data Ingestion & ML), scikit-learn (Anomaly Detection), LLM API (OpenAI/Anthropic/Local LLaMA for Triage), SQLite/Elasticsearch (Log Storage), FastAPI (Backend Web Server).

Business Context: Security Operations Centers (SOCs) across West Africa's energy and banking sectors suffer from "Alert Fatigue," missing zero-day threats in a sea of false positives. Aegis AI automates the Tier-1 analyst role by translating raw network anomalies into plain-English remediation playbooks.

> **Project Scope:** An end-to-end B2B security intelligence platform. The system ingests raw server logs, runs unsupervised machine learning to detect statistical outliers, feeds those outliers into an LLM for natural language triage, and outputs a threat assessment.
> **Target Audience:** Chief Information Security Officers (CISOs), Enterprise IT Managers, and Security Engineering teams at firms like Shell, local banks, or telecoms.

1. System Architecture Blueprint

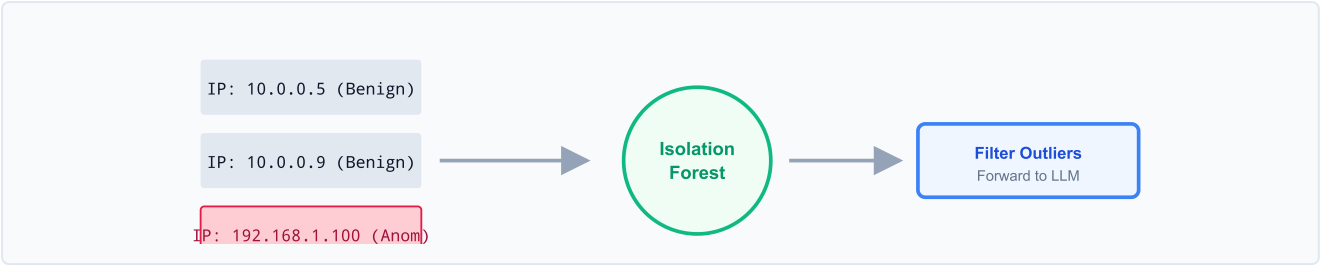


2. Core Modules & Data Pipeline

Module	Technology	Responsibility
Ingestion Engine	Python (Pandas)	Simulates and normalizes incoming network traffic JSON logs. Extracts IP, ports, byte count, and login flags.
Detection Core	Isolation Forest (ML)	Unsupervised learning algorithm designed specifically for anomaly detection. Flags statistical outliers.
AI Translator	LLM API (OpenAI)	Receives the outlier JSON. Outputs a structured, human-readable threat summary mapping to MITRE ATT&CK tactics.
API Gateway	FastAPI	Serves the finalized threat reports to the frontend UI dashboard for human approval.

3. Log Generation & The ML Pipeline

AI INSTRUCTION: Because you do not have live enterprise network traffic to train on, you must write a Python script to synthesize realistic mock logs. The script should generate 99% "benign" background traffic and 1% "malicious" anomalies (e.g., impossible travel, brute-force login attempts, massive data exfiltration spikes).



Data Structure for Log Generation

Ensure the Python generator outputs a JSON structure similar to this. Include geographic variation to simulate multi-region infrastructure (e.g., VPN connections originating from Port Harcourt versus unexpected foreign IPs).

```
{ "timestamp": "2026-10-09T14:32:01Z", "source_ip": "45.22.11.90", "dest_ip": "10.0.4.15", "dest_port": 22, "protocol": "TCP", "bytes_transferred": 1054300, "failed_login_attempts": 12, "user_agent": "Nmap Scripting Engine", "geo_location": "RU" }
```

> **Machine Learning Constraint:** Do not build complex deep learning models for the MVP. Use `sklearn.ensemble.IsolationForest`. It is computationally lightweight, perfectly suited for outlier detection in tabular log data, and proves you understand practical, deployable engineering over academic complexity.

4. LLM Triage & The Prompt Architecture

AI INSTRUCTION: Once the Isolation Forest flags a log as an anomaly (e.g., `anomaly_score = -1`), the system must immediately pass that JSON record to an LLM via API. The agent must engineer a strict system prompt to force the LLM to return structured, deterministic JSON playbooks, avoiding "chatty" responses.

The System Prompt Formula

You are building a specialized agent. Use the following structure to configure the LLM behavior within your Python backend code:

```
SYSTEM_PROMPT = """ You are Aegis, an expert Tier-3 Cybersecurity Analyst. I will provide you with a raw network log flagged as an anomaly. Your job is to: 1. Identify the likely MITRE ATT&CK tactic (e.g., Initial Access, Exfiltration). 2. Assess the severity (Low, Medium, High, CRITICAL). 3. Write a 2-sentence plain English summary for a junior analyst. 4. Provide 3 actionable remediation steps. Respond ONLY in valid JSON matching this schema: {"tactic": "", "severity": "", "summary": "", "remediation": []} """
```

> **Prompt Engineering Note:** By forcing the LLM to output structured JSON, your FastAPI backend can easily parse the response and serve it to the frontend dashboard. If the LLM returns conversational text (e.g., "Sure, here is the analysis..."), the UI will break.

Expected Input-Output Mapping

Flagged ML Outlier (Input)	LLM Playbook (Output)
`dest_port: 3389, failed_logins: 450, geo: "Unknown"`	**Tactic:** Brute Force (Credential Access) **Severity:** HIGH **Remediation:** 1. Block source IP at edge firewall. 2. Force password reset for target user.
`dest_port: 443, bytes: 5000000000, time: 03:00AM`	**Tactic:** Data Exfiltration **Severity:** CRITICAL **Remediation:** 1. Sever external connection. 2. Audit DB access logs for last 2 hours.

5. AI Execution Checklist

AI INSTRUCTION: Generate the codebase sequentially following these phases. Validate dependencies (like Pandas and Scikit-Learn) before writing execution scripts.

Phase	Component	Success Criteria
Phase 1	Log Generator Script	Creates a `.csv` or SQLite DB with 10,000 benign records and 50 injected malicious records.
Phase 2	ML Training Pipeline	Trains an `IsolationForest`. Accurately flags the 50 injected anomalies without excessive false positives.
Phase 3	LLM Integration Module	Python function that accepts an outlier row, calls the OpenAI API (with the strict System Prompt), and returns JSON.
Phase 4	Dashboard API (FastAPI)	Exposes a `/api/alerts/` endpoint serving the enriched LLM threat playbooks to a frontend.

Monetization & Interview Pitch Strategy

When presenting Aegis AI to potential employers or enterprise clients in the energy and banking sectors, focus strictly on unit economics and operational metrics. Do not get bogged down explaining how the LLM works; explain how much money the LLM saves.

> **The "MTTR" Business Case:**
> Security leaders care about one metric above all: *Mean Time To Respond (MTTR)*. Explain that a human analyst takes 15–30 minutes to investigate a single suspicious log to determine if it is a false positive. Aegis AI ingests, detects, translates, and drafts a response playbook in **under 3 seconds**. By reducing MTTR and eliminating alert fatigue, you are mathematically preventing multi-million dollar data breaches caused by human oversight.

Final Agent Directive: Begin by generating the Python data synthesis script (Phase 1). Ensure standard network protocol mapping is accurate.